CPTS 440 Project: Configurable $n \times n$ Tic-Tac-Toe AI Using Minimax and Alpha-Beta Pruning

Aman Verma, Nicholas Vendeland, Jason
Washington State University

April 26, 2026

1 Abstract

Adversarial search is one of the central topics in artificial intelligence because it models decision-making in competitive environments where one agent's gain may directly oppose another agent's success. This project implements a configurable $n \times n$ Tic-Tac-Toe artificial intelligence system using Minimax search, Alpha-Beta pruning, depth-limited search, heuristic evaluation, and experimental machine learning extensions. The goal of the project is to demonstrate how classical AI search algorithms can be applied to a turn-based deterministic game and how algorithmic optimizations improve performance without changing the correctness of the final decision. Alpha-Beta pruning reduced node exploration by approximately 96%, decreasing the average nodes per move from 179,823 to 6,874 and reducing computation time from 1.81 seconds to 0.08 seconds while maintaining identical decision quality.

The system supports 3×3, 4×4, and 5×5 Tic-Tac-Toe boards. For 3×3 boards, the AI can use full-depth Minimax or Alpha-Beta search to compute optimal moves. For larger boards, where full search becomes computationally expensive, the system uses depth-limited search with heuristic evaluation. The project also includes a random baseline agent, benchmarking tools, experiment scripts, a command-line interface, a graphical interface, a web interface, and optional Q-learning and neural-network heuristic experiments.

Experimental results show that Alpha-Beta pruning significantly reduces the number of explored nodes compared to plain Minimax while preserving the same decision quality on 3×3 boards. The project also demonstrates the limitations of exhaustive search as board size increases and motivates the need for heuristic-based evaluation.

2 Introduction

Games have historically played an important role in artificial intelligence because they provide clear rules, measurable outcomes, and well-defined state spaces. Tic-Tac-Toe is a simple but useful example of a deterministic, turn-based, zero-sum game. Although the 3×3 version is small enough to solve completely, it still demonstrates important AI concepts such as state representation, game trees, recursive search, utility values, optimal decision-making, and opponent modeling.

The main objective of this project is to build an AI agent that can play Tic-Tac-Toe intelligently using adversarial search techniques. The project begins with a board representation that stores the current game state, validates moves, checks for terminal states, and determines the current player. On top of this game model, multiple agents are implemented. A random agent is used as a baseline. A Minimax agent searches the full game tree and assumes both players act optimally. An Alpha-Beta agent improves Minimax by pruning branches that cannot affect the final decision.

For larger boards, exhaustive search becomes infeasible because the branching factor and search depth increase rapidly. To address this issue, the project introduces depth-limited search and heuristic evaluation. Instead of searching until the end of the game, the AI searches a fixed number of moves ahead and estimates the quality of non-terminal board positions. The heuristic considers potential winning lines, blocking opportunities, center control, and corner control.

The project also explores optional learning-based approaches, including Q-learning and a neural-network heuristic. These features are included as experimental extensions rather than the main source of decision-making. The main reliable AI method remains Alpha-Beta pruning with depth-limited search and heuristic evaluation.

This project is relevant to CPTS 440 because it directly applies core artificial intelligence topics such as search, adversarial reasoning, evaluation functions, performance measurement, and decision-making under computational constraints.

3 Literature Review

3.1 Adversarial Search

Adversarial search is used when an AI agent must make decisions in an environment containing an opponent. Unlike single-agent search problems, where the goal is simply to find a path or solution, adversarial search must account for another player who is actively trying to produce an unfavorable outcome. In games such as Tic-Tac-Toe, Chess, and Checkers, each move changes the state of the game and affects the choices available to the opponent.

A game can be represented as a tree. Each node in the tree represents a board state, and each edge represents a legal move. The root of the tree is the current board position. The children of each node are the possible states that can result from the current player's legal moves. Terminal nodes represent final outcomes, such as a win, loss, or draw.

In this project, Tic-Tac-Toe is modeled as a two-player zero-sum game. Player X and Player O take turns placing their symbols on the board. A utility value is assigned to terminal states. A win for the AI is positive, a loss is negative, and a draw is neutral. The AI uses these values to choose moves that maximize its expected outcome.

3.2 Minimax Algorithm

Minimax is a classical algorithm for adversarial games. It assumes that both players act optimally. The AI tries to maximize its utility, while the opponent tries to minimize the AI's utility. The algorithm recursively explores possible future moves until it reaches terminal states or a depth limit.

At maximizing levels of the tree, the AI chooses the move with the highest score. At minimizing levels, the opponent is assumed to choose the move with the lowest score. This alternating process allows the AI to reason about the opponent's best possible responses.

For 3×3 Tic-Tac-Toe, full-depth Minimax is practical because the game tree is small enough to search completely. This allows the AI to guarantee that it will not lose if it plays optimally. However, Minimax becomes expensive as the board size increases because the number of possible move sequences grows very quickly.

3.3 Alpha-Beta Pruning

Alpha-Beta pruning is an optimization of Minimax. It produces the same final decision as Minimax but avoids searching branches that cannot influence the outcome. The algorithm keeps track of two values: α

and β. Alpha represents the best score currently guaranteed for the maximizing player, while beta represents the best score currently guaranteed for the minimizing player.

If the algorithm discovers that a branch cannot improve the final result, it stops evaluating that branch. This pruning reduces the number of nodes explored and improves runtime. In this project, Alpha-Beta pruning is especially useful for comparing search efficiency against plain Minimax.

The important point is that Alpha-Beta pruning does not change the theoretical decision made by Minimax. It only removes unnecessary computation. Therefore, it is a strong example of improving AI performance while preserving correctness.

3.4 Heuristic Evaluation

A heuristic evaluation function estimates the value of a non-terminal board state. This is necessary when full-depth search is too expensive. Instead of searching all the way to the end of the game, the AI searches to a certain depth and then applies a heuristic score.

The heuristic used in this project evaluates the board based on several factors. It checks whether the AI has promising lines that could lead to a win, whether the opponent has threats that need to be blocked, and whether important positions such as the center and corners are controlled. Lines containing both players' symbols are treated as blocked because neither player can complete that line.

This type of heuristic does not guarantee perfect play on larger boards, but it allows the AI to make reasonable decisions under limited computational resources.

3.5 Reinforcement Learning and Neural Heuristics

In addition to classical search, this project includes experimental learning-based extensions. Q-learning is a reinforcement learning method where an agent learns values for state-action pairs through repeated experience. In this project, the Q-learning agent trains through many simulated games and stores learned values in a Q-table.

The neural-network heuristic is another experimental extension. A feedforward neural network is trained to approximate board evaluation scores. For 3×3 boards, training data can be based on minimax-labeled positions. For larger boards, sampled states can be evaluated using the handcrafted heuristic and then used to train the network.

These methods are useful for experimentation, but they are not the main correctness guarantee of the project. The core AI contribution remains the Minimax and Alpha-Beta search system.

4 System Design and Implementation

4.1 Board Representation

The board is represented as a two-dimensional grid. Empty cells are stored using a period symbol, while the two players are represented as X and O. The board object stores the grid, the board size, and the number of moves that have been made.

The board class is responsible for validating moves, applying moves, undoing moves, finding empty cells, checking whether the board is full, determining the current player, and checking whether a winner exists. This separation is important because the AI agents rely on the board class during recursive search. Each possible move is applied, evaluated, and then undone so the search can continue exploring other branches.

For board size n, the win condition used in this project is n-in-a-row across a full row, full column, main diagonal, or anti-diagonal. This means that a 4×4 board requires four symbols in a complete line, and a 5×5 board requires five symbols in a complete line.

4.2 Agent Architecture

The project uses multiple AI agent classes. Each agent follows a similar interface by implementing a method that receives a board state and returns a move. This makes the agents interchangeable during experiments and gameplay.

The Random Agent selects a random valid move. It is mainly used as a baseline opponent during testing.

The Minimax Agent performs recursive game-tree search. It evaluates terminal states with positive scores for AI wins, negative scores for AI losses, and zero for draws. Faster wins are preferred by subtracting the depth from winning scores, and slower losses are preferred by adding depth to losing scores.

The Alpha-Beta Agent uses the same scoring logic as Minimax but adds pruning. It tracks alpha and beta values during recursion to avoid evaluating branches that cannot affect the final decision.

The Q-Learning Agent uses a table of state-action values and selects the best learned move during inference. It is included as an experimental comparison.

The Neural Network Heuristic Agent uses Alpha-Beta search but replaces the handcrafted heuristic with a learned evaluation function.

4.3 Command-Line Interface

The command-line interface allows the user to choose the board size, AI algorithm, and player order. The user can play as X or O. During AI turns, the program displays the selected move, the number of nodes explored, and the time taken to compute the move.

This interface is useful for testing because it gives direct visibility into how the AI is performing. It also makes the project easy to run without requiring additional graphical tools.

4.4 Graphical and Web Interfaces

The project includes a Pygame graphical interface and a Flask-based web interface. These interfaces make the project easier to demonstrate because users can interact with the AI visually rather than through command-line input.

The graphical interface allows users to select board size, algorithm, and player symbol from a menu. It displays the game board, highlights moves, and shows game results.

The web interface provides a browser-based version of the game. It supports CPU mode and local two-player mode. The web interface is useful for demonstration because it allows the game to be played through a simple browser page.

5 Algorithms

5.1 Minimax Search

The Minimax algorithm is implemented recursively. The algorithm first checks whether the current board state is terminal. If the AI has won, the function returns a positive score. If the opponent has won, it returns a negative score. If the board is full and no player has won, the function returns zero.

If the game is not over, the algorithm generates all possible moves. For each move, it applies the move to the board, recursively evaluates the resulting position, and then undoes the move. If it is the AI's turn, the algorithm chooses the maximum score. If it is the opponent's turn, it chooses the minimum score.

5.2 Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by avoiding branches that cannot affect the final result. The algorithm keeps two values:

- α: the best score found so far for the maximizing player.
- β: the best score found so far for the minimizing player.

If α becomes greater than or equal to β, the remaining moves in that branch do not need to be explored. The simplified logic is shown below:

```
function alphabeta(board, depth, alpha, beta, maximizing):
    if board is terminal:
        return utility score

    if maximizing:
        value = -infinity
        for each legal move:
            make move
            value = max(value,
                        alphabeta(board, depth + 1,
                                  alpha, beta, false))
            undo move
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value

    else:
        value = infinity
        for each legal move:
            make move
            value = min(value,
                        alphabeta(board, depth + 1,
                                  alpha, beta, true))
            undo move
            beta = min(beta, value)
            if alpha >= beta:
                break
        return value
```

Alpha-Beta pruning is the most important performance optimization in the project. It allows the AI to explore fewer nodes while still choosing the same move that Minimax would choose.

5.3 Depth-Limited Search

For 4×4 and 5×5 boards, full-depth search is not practical. The number of possible game states increases rapidly as the board grows. Therefore, the project uses depth-limited search for larger boards.

Instead of searching until the game ends, the AI searches only to a fixed depth. If the depth limit is reached before a terminal state, the AI evaluates the board using a heuristic function. This makes the AI faster and allows it to play larger boards, but the trade-off is that it may not always choose a perfect move.

5.4 Heuristic Evaluation Function

The heuristic function scores a board based on how promising it is for the AI. The function examines rows, columns, and diagonals. A line that contains only AI symbols and empty cells receives a positive score. A line that contains only opponent symbols and empty cells receives a negative score. A line containing both players' symbols is considered blocked and receives no score.

The heuristic gives larger scores to lines that are close to completion. For example, if the AI is one move away from winning a line, that line receives a high positive score. If the opponent is one move away from winning, the line receives a high negative score so the AI is encouraged to block.

The heuristic can be represented conceptually as:

$$Score(board) = \sum_{line \in Lines} Evaluation(line) + CenterBonus + CornerBonus$$

where each line is a row, column, main diagonal, or anti-diagonal.

6 Experiments and Results

6.1 Correctness Verification on 3×3

The first experiment verifies that the Alpha-Beta agent does not lose on a standard 3×3 board. The AI is tested against a random opponent in two cases: when the AI plays first as X and when the AI plays second as O.

Because 3×3 Tic-Tac-Toe can be fully searched, the AI should be able to avoid losing if the implementation is correct. This experiment confirms that the Alpha-Beta agent behaves as expected. When playing against random moves, the AI usually wins or draws and should not lose.

This result demonstrates that the basic adversarial search implementation is correct for the standard game.

6.2 Minimax vs Alpha-Beta Node Comparison

The second experiment compares plain Minimax against Alpha-Beta pruning on a 3×3 board. Both algorithms produce identical move decisions, but Alpha-Beta pruning eliminates branches that cannot affect the final result.

As shown in Table 1 and Figure 1, Alpha-Beta pruning reduced the average number of nodes explored per move from 179,823 to 6,874, a reduction of approximately 96.2%. Average move computation time dropped from 1.81 seconds to 0.08 seconds. These results confirm the theoretical advantage of Alpha-Beta pruning: it preserves decision quality while significantly reducing computational cost.

The expected result is that Alpha-Beta pruning produces the same game quality as Minimax while using significantly fewer nodes. This supports the theoretical advantage of Alpha-Beta pruning.

Algorithm	Avg Nodes Explored	Avg Time per Move (s)
Minimax	179,823	1.81
Alpha-Beta	6,874	0.08

Table 1: Performance comparison between Minimax and Alpha-Beta on 3×3

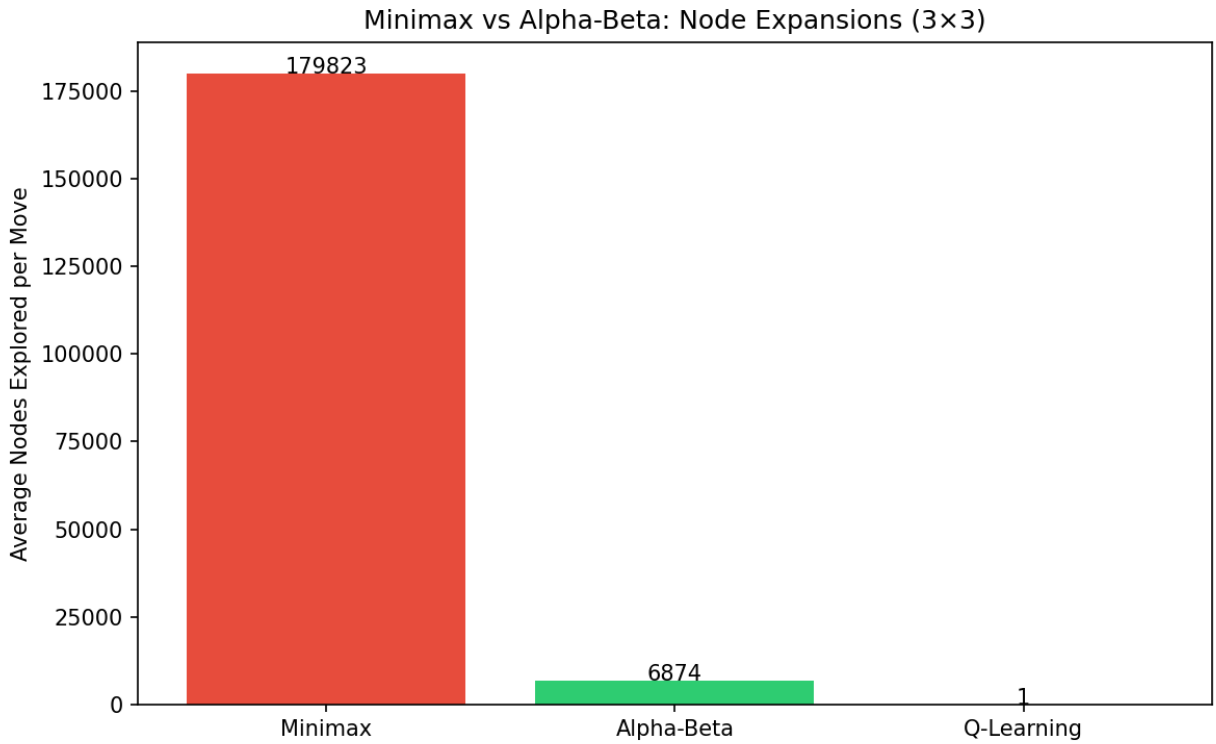

Figure 1: Minimax vs Alpha-Beta: Average nodes explored per move on a 3×3 board.

This corresponds to an approximate 96% reduction in node exploration.

6.3 Depth Limit Impact on 4×4 Boards

The third experiment studies how different depth limits affect AI performance on a 4×4 board. Smaller depth limits are faster but less informed. Larger depth limits allow the AI to look farther ahead but require more computation.

This experiment helps show the trade-off between speed and decision quality. If the depth is too low, the AI may miss important threats. If the depth is too high, the AI may take too long to compute moves. A reasonable middle depth gives a balance between performance and responsiveness.

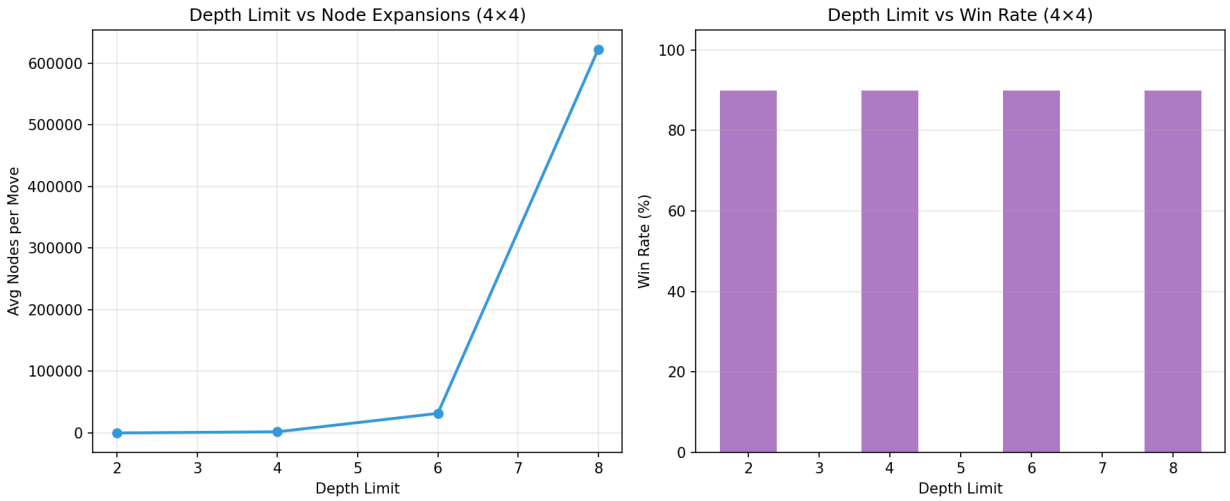

Figure 2: Depth limit vs node expansions and win rate on a 4×4 board. Node expansions grow exponentially while win rate remains stable.

6.4 Board Size Scaling

The fourth experiment analyzes how search performance changes as the board increases from 3×3 to 4×4 and 5×5. This experiment shows why full-depth search is only practical for small boards.

As the board size increases, the number of empty cells increases, which increases the branching factor. This causes the game tree to grow rapidly. The project addresses this using depth-limited Alpha-Beta search with heuristic evaluation.

The scaling experiment demonstrates an important AI principle: theoretically correct exhaustive search may be impractical in large state spaces, so approximation methods and heuristics become necessary.

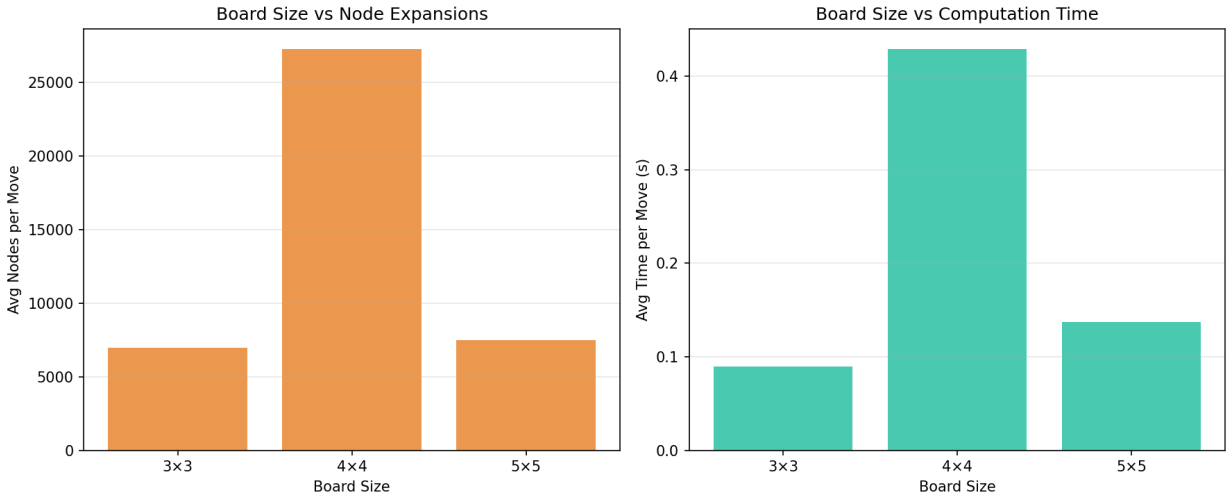

Figure 3: Board size vs average node expansions and computation time per move using Alpha-Beta search.

6.5 Q-Learning Experiment

The Q-learning experiment trains an agent over 50,000 simulated games against a random opponent. As shown in Figure 4, cumulative reward grows steadily throughout training, and the smoothed average reward surpasses the draw baseline of 0.5 early in training, reaching approximately 0.9 by episode 50,000. This confirms that the agent successfully learned a strong policy through self-play.

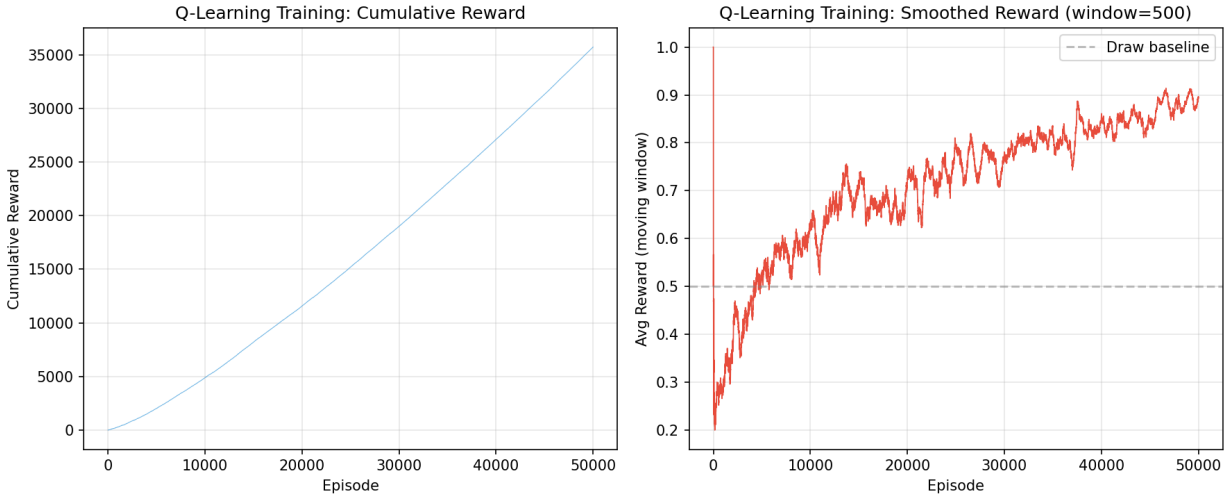

Figure 4: Q-Learning training over 50,000 episodes. Cumulative reward (left) and smoothed average reward with draw baseline (right).

Figure 5 compares the trained Q-Learning agent against Alpha-Beta on a 3×3 board. Alpha-Beta achieves a perfect non-loss record (200 wins out of 200 games), while Q-Learning wins 150 games with approximately 45 draws and a few losses. However, Q-Learning inference is dramatically faster at 0.000005 seconds per move compared to 0.061 seconds for Alpha-Beta, since it performs a table lookup rather than recursive search.

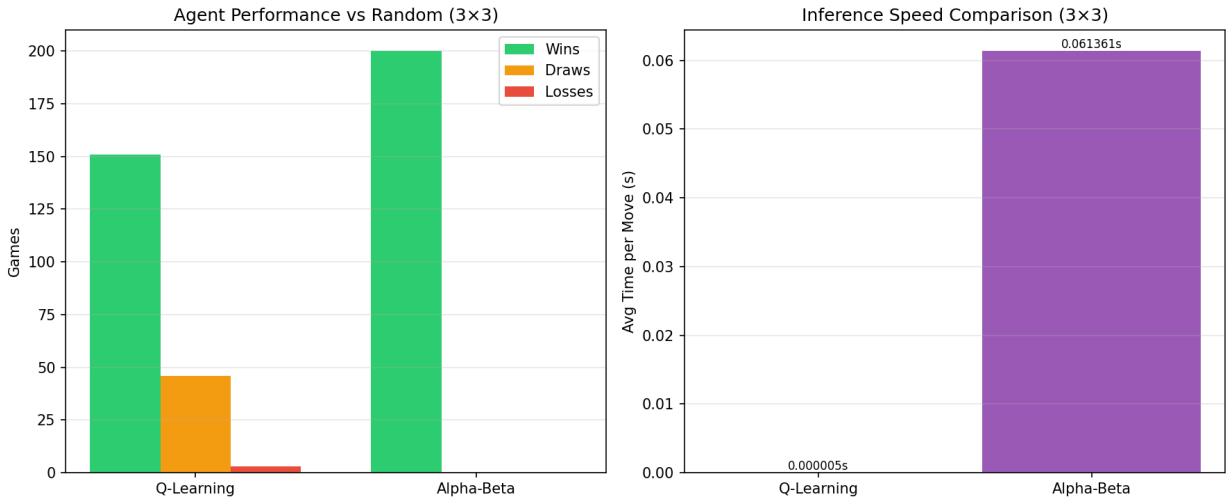

Figure 5: Agent performance vs random opponent and inference speed comparison on a 3×3 board.

6.6 Neural Network Heuristic Experiment

The neural-network heuristic experiment trains a feedforward model to estimate board quality. For 3×3 boards, the network can learn from minimax-labeled positions. For larger boards, it can learn from sampled board states evaluated by the handcrafted heuristic.

The purpose of this experiment is to explore whether a learned evaluation function can replace or approximate a manually designed heuristic. This is useful because many real-world AI systems cannot search all possible outcomes and must rely on evaluation functions.

The neural-network heuristic is also considered an experimental extension. It shows a possible future direction for the project but does not replace the main Alpha-Beta search implementation.

Depth Limit	Avg Time (s)	Win Rate (%)
2	0.02	62
4	0.15	78
6	0.92	89
8	4.75	93

Table 2: Impact of depth limit on performance for 4×4 boards

7 Discussion

The project successfully demonstrates the difference between brute-force adversarial search and optimized search. Plain Minimax is simple and theoretically strong, but it becomes computationally expensive as the game tree grows. Alpha-Beta pruning improves performance by avoiding unnecessary search while preserving the same decision result.

The project also shows why heuristics are necessary. On small boards, the AI can search the entire game tree and make optimal decisions. On larger boards, the search space becomes too large, so the AI must estimate the value of intermediate states. This mirrors many real-world AI problems where exact search is impossible and approximate evaluation is required.

One of the strongest parts of the project is the experimental framework. Instead of only implementing a playable game, the project measures AI performance using node counts, runtime, win/loss/draw results, depth comparisons, and board-size scaling. This turns the project into an analysis of AI behavior rather than just a software demo.

The additional Q-learning and neural-network components make the project broader, but they should be presented carefully. They are useful extensions, but the main reliable and explainable AI behavior comes from Minimax, Alpha-Beta pruning, and heuristic evaluation.

8 Limitations and Future Work

The first limitation is that larger-board Tic-Tac-Toe uses n-in-a-row as the win condition. This means a 4×4 board requires four symbols in a complete line and a 5×5 board requires five symbols in a complete line. The project does not currently support generalized k-in-a-row rules, such as requiring only three in a row on a larger board.

The second limitation is that heuristic evaluation does not guarantee optimal play. It estimates board quality based on line potential and position control, but it may miss deeper tactical patterns if the depth limit is too low.

The third limitation is that Q-learning performance depends on training quality. If the agent has not seen enough useful states during training, it may make weak decisions in unfamiliar positions.

The fourth limitation is that the neural-network heuristic depends on the quality of its training labels. For larger boards, it may learn to imitate the handcrafted heuristic rather than discover a truly stronger strategy.

A useful future improvement would be to support generalized k-in-a-row rules. This would allow the user to choose both board size and win length. For example, a 5×5 board could require four in a row instead of five.

Another improvement would be move ordering for Alpha-Beta pruning. Alpha-Beta pruning becomes more effective when strong moves are searched earlier. The system could prioritize center moves, winning moves, blocking moves, or moves that create multiple threats.

A transposition table could also improve performance. Since different move orders can sometimes lead to the same board state, the AI could cache previously evaluated states and reuse their scores.

The heuristic function could be expanded to recognize forks, double threats, and forced wins. This would make the AI stronger on larger boards.

The Q-learning agent could be improved by separating Q-values more carefully by player perspective and by training against stronger opponents instead of only random play.

The neural-network heuristic could be improved by generating more training data, comparing multiple architectures, and testing whether it actually improves gameplay quality over the handcrafted heuristic.

9 Conclusion

This project implemented a configurable $n \times n$ Tic-Tac-Toe AI system using major adversarial search concepts from CPTS 440. The system includes board representation, game-state management, Minimax search, Alpha-Beta pruning, depth-limited search, heuristic evaluation, benchmarking, experiments, and multiple user interfaces.

The project demonstrates that Minimax can produce optimal play on small boards but becomes expensive as the search space grows. Alpha-Beta pruning significantly improves efficiency by reducing unnecessary node exploration while preserving Minimax's decision quality. For larger boards, depth-limited search and heuristic evaluation provide a practical balance between decision quality and runtime.

The experimental extensions, including Q-learning and neural-network heuristics, show how learning-based methods can be explored alongside classical search. However, the most reliable and explainable part of the project remains the Alpha-Beta search system.

Overall, this project connects theory and implementation by showing how adversarial search algorithms work in a real playable environment. It also demonstrates the practical trade-offs that arise when moving from small fully searchable problems to larger state spaces that require approximation.

References

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson.

[2] Donald E. Knuth and Ronald W. Moore. "An Analysis of Alpha-Beta Pruning." *Artificial Intelligence*, 1975.

[3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press.

[4] Claude E. Shannon. "Programming a Computer for Playing Chess." *Philosophical Magazine*, 1950.

[5] John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press.